

Common Mistakes Naming Routines in EECS 293

Note: these naming conventions are the conventions used in EECS 293. Other environments may use different conventions, but regardless it is important to be consistent within an environment so your fellow programmers can tell what your code does.

Mistaking procedures and functions

Remember that a procedure is primarily designed to perform an action (such as changing state, displaying information, etc.). They *may* return values, but those are not the primary purpose. The return value is usually some information about what was done, a status flag, or similar.

A function is primarily designed to return a value. If it makes any changes, those should not be noticeable to someone using a function and should not be the primary purpose of the routine. Such a change may be caching a value.

Including verbs in function names

Probably the most common mistake in this class is including a verb in a function name. If a routine is a function, then it should *only* have what it returns in its name, and not how it gets it. For example, for a function that sums numbers, it should be called `sum()` and not `calculateSum()`, since the latter would make it look like a procedure of some kind. Likewise, a function that gets the minimum value stored in a set should be called `min()` and not `findMin()`.

Similarly, if you have a function that is designed to return a new object, call it what it returns. For example, an object that returns a new Flight object should be called something like `newFlight()` instead of `generateNewFlight()`.

Inappropriately calling a function a getter

A function should *only* have “get” in its name if it returns a private variable or something that could be considered essentially a private variable (such as an entry in a private map). If it performs any sort of computation to return the value, then the word “get” should not be in the name.

For example, in the air travel project where the departure time is computed by adding the connection time to the arrival time, the value returned is the result of a computation. Thus, the function that returns the departure time should be called `departureTime()` and not `getDepartureTime()`.

Including unnecessary information

Only the important details of a routine name should be included in its name. Less important details should not be included.

This happens most commonly when naming procedures, as often the return value is included. For example, a function that adds a flight and returns a status variable indicating whether or not it could be added should just be called `add()` instead of `addAndStatus()`. Likewise, something that adds multiple flights and returns the number that were successfully added should be called `addAll()` instead of `addAllAndNumAdded()`.

Similarly, conditions on the input should not be included unless they are important to the primary intent of the routine. For example, a square root function that rejects negative numbers should be called `sqrt()` and not `sqrtRejectsNegative()`.

Naming routines by common phrases that don't describe what they actually do

It is often convenient to use commonly used words or words that mirror other functions to name your routines. However, if you do that, make sure that actually describes what your routine does and isn't just something that has a similar idea. For example, if you are seeing if a `RouteNode`'s airport matches an `Airport` object (when `RouteNode` stores an airport), it may be tempting to call your function `equals()` in keeping with the function for all Java objects. However, if you think about what you're actually doing, a `RouteNode` cannot equal an airport, so a better name would be `airportMatches()`, since that actually clearly explains that you're comparing the `Airport` within the `RouteNode`.

Including objects unnecessarily

Often, the object that a function references (whether it's an argument or part of the state) is implied. In that case, you do not need to include it in the function name. For example, a function that adds a `RouteNode` could easily be called `add()` instead of `addRouteNode()` since the fact that the argument is a `RouteNode` is implied by the argument. Also, the function on a `Route` object to display it should be called `display()` instead of `displayRoute()` since the fact that it's displaying the `Route` is implied by the type of the object, so including it in the function name is redundant. Another example is that if you have an `ArrivalTime` object and have a method to add another `ArrivalTime` to it, you can just call it `add()` since both sides of what are being added are implied (by the fact that it and its argument are both `ArrivalTimes`), so calling it `addToArrivalTime()` or `addArrivalTime()` (or even worse, `addArrivalTimeToArrivalTime()`) would all be redundant.

Using non-descriptive words

Words like "do", "handle", "process", etc. should be avoided in routine names at all costs. Those words do not in any way describe what the routine actually does, so including them adds length to the function name with no actual information. If you can't come up with a more descriptive name, that's probably a sign that your routine is not cohesive and needs to be rewritten. In most cases, this is because you are cramming too much into a single routine and it needs to be broken up into multiple routines.

For example, a function that cycles through a set of `Flights` and replaces all `RouteNode` objects with new `RouteNodes` with arrival times if the `Flight` has an earlier arrival time, it should be called something like `replaceNodesWithEarlierArrivals()` instead of `processFlights()`.

A routine that needs to be rewritten would be one that computes quarterly financial totals, annual totals, next year's forecast, and then saves them to a report file. A naïve name would be `analyzeFinances()`, while it should be split up into `quarterlyTotals()`, `annualTotals()`, `nextYearForecast()`, and `saveToReportFile()`.